

Plan, Gate, Learn: A Verified Multi-Agent Orchestrator for Natural-Language Cloud Data Pipelines

Manvith M. Nayak¹ and Co-Author Name¹

Department / Institution, City, Country
manvithnayak8704@gmail.com

Abstract. Describing a data pipeline takes one sentence; building it on a cloud platform takes an afternoon of containers, copy activities, Spark code and compute settings. We built an orchestrator in which nine agents turn a CSV file and a plain-English request into a running Azure Data Factory and Databricks pipeline, and we measured where each agent earns its place. A 7B model fine-tuned with QLoRA understands 96% of unseen requests, against 33% before tuning, but only a deterministic repair step makes its plans runnable (33% to 100%); a 120B cloud model given the same rules is correct on 53%. A structural gate stops every one of 1,400 injected plan faults before the cloud, two thirds of which would otherwise have produced wrong data without any error. On live Azure runs, a fix to the learning loop cut duration-estimate error from 53.6% to 14.4%, and learned corrections cut cost-estimate error by 41–47%. We also report what went wrong: a learned abort rule that is right only 54% of the time, and a silent data corruption that only output checks caught.

Keywords: LLM agents · data pipelines · fine-tuning · verification · self-adaptive systems

1 Introduction

Suppose an analyst wants the average order value per region, but only for orders with more than five items. On Azure this means creating storage containers, a linked service, datasets and a Data Factory copy pipeline, writing and debugging a Databricks notebook of 100–200 lines, and picking throughput units, workers and shuffle partitions that fit the subscription. Mistakes show up late. A filter on a misspelled column compiles without complaint and fails inside Spark, after the cluster has already been paid for.

Language models can write such a configuration, and that is where the trouble starts. A plan that is almost right does not run. A plan that runs may quietly answer a different question. And nothing in the model knows what a run will cost or how long it will take. Our orchestrator treats all three as separate problems, gives each decision to exactly one agent, and puts a deterministic check behind every learned component.

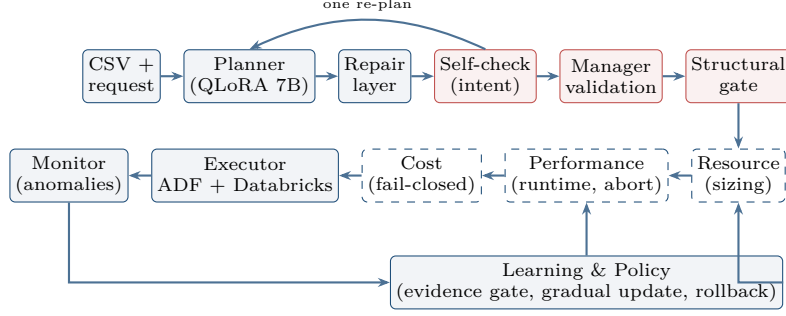


Fig. 1. Life of a run. Solid boxes are deterministic, dashed boxes are learned, and red boxes can stop a run before any cloud spend.

The paper makes four contributions: (i) a planner that combines a small fine-tuned model, deterministic repair and a single self-verification step, and beats a 120B cloud model on the same contract (Sect. 5.1); (ii) layered gates, evaluated with 1,400 injected faults, that keep wrong plans out of the cloud (Sect. 5.2); (iii) a closed learning loop with evidence gates and automatic rollback, evaluated live and in simulation (Sect. 5.4); and (iv) a live Azure evaluation in which every output was checked against a reference, including the failures (Sect. 5.3, 5.5).

2 Related Work

Agents that reason and act [5], reflect [6] or refine their own output [7], and frameworks that split work across agents [9,10], make LLMs more capable on multi-step tasks. Without outside feedback, though, models rarely fix their own reasoning [8], which is why our planner only re-plans on feedback from checks it cannot argue with. Text-to-SQL work [11,12] shows strong but brittle generation of single queries; a pipeline plan also fixes storage, stage wiring and resources, so small mistakes stop it from running at all. LoRA [2] and QLoRA [3] make it cheap to specialise a base model such as Qwen2.5 [4]. Learned tuning of data systems [13] and the MAPE-K loop of autonomic computing [1] shaped our predictors and learning loop; we add evidence gating and rollback, and show where rollback itself goes wrong.

3 System

Figure 1 shows the path of a request. The Planner designs the pipeline, the Resource Agent picks compute settings within the subscription’s limits, the Performance Agent forecasts runtime and may abort a run it expects to fail, the Cost Agent looks for cheaper settings, and the Executor deploys the copy and runs the notebooks. The Central Manager only sequences these agents and enforces their verdicts. Each decision has one owner. This rule came from experience: in

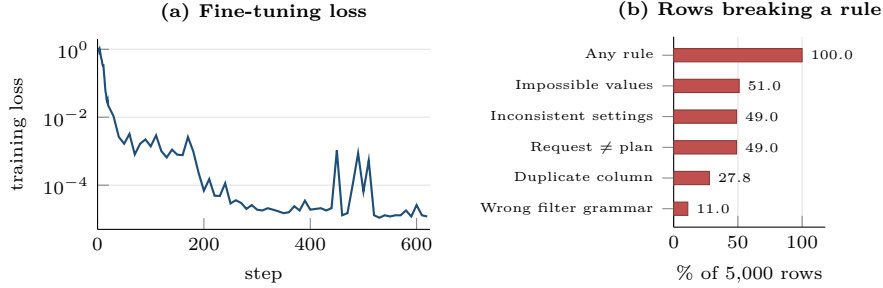


Fig. 2. (a) Training loss over 625 steps (one epoch, T4): the plan grammar is learned within about 100 steps, which is why we test on requests phrased unlike the training data. (b) Share of rows in the earlier template dataset that break each rule; the validated dataset breaks none.

an earlier version the runtime model’s strongest input (importance 0.91) was the Resource Agent’s own runtime guess, so two agents were silently predicting the same thing. Independent stages run concurrently as *execution groups*, and streaming plans process only new files on each tick.

Planner. We generated 5,000 training examples (schema, request, plan) across 18 domains and 2–5 stages. An independent validator enforces 19 rules covering structure, value ranges, one filter grammar and semantic sanity, such as no contradictory filters. Every row passes. An earlier template-based dataset failed at least one rule in every row (Fig. 2b). We tuned Qwen2.5-7B-Instruct [4] with QLoRA [3] (4-bit base, rank 16) on a single Tesla T4 in 4.5 hours, training 0.53% of the parameters; the adapter is an 81 MB file served locally. Each raw plan then goes through a deterministic repair layer (Azure-safe names, stage wiring, operator types, size-aware caps) and a self-check: the structural rules plus an intent check by a separate, untuned 7B model. If the check finds a problem, the planner re-plans once with that problem as feedback.

The intent checker needed help. Given only the request and raw JSON, it invented column names and flagged correct plans. Once it saw the real columns and a plain-language description of each stage, and two simple rules discarded complaints about non-existent columns or changes the plan already made, its accuracy on a fixed test suite rose from 15/18 to 27/27.

Learning loop. Every five runs, the Learning & Policy Agent compares predictions with what actually happened. A correction factor changes only after at least 10 relevant runs, moves 30% of the way toward the observed ratio, and is reverted if error rises by more than 5% over the next five runs. When runtime error passes 20%, the runtime model is retrained with real runs blended in and kept only if held-out error does not get worse.

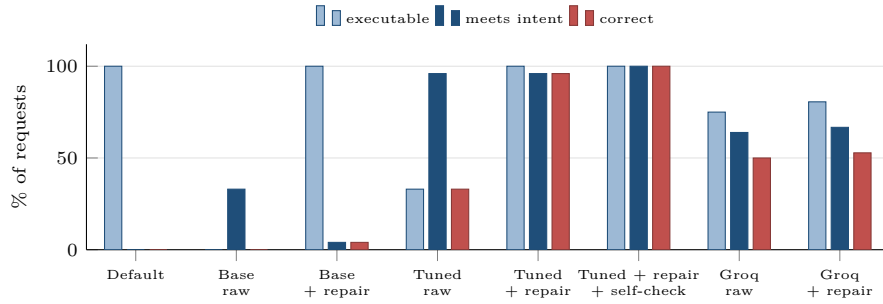


Fig. 3. Planner ablation on 24 unseen requests. “Default” is the deterministic fallback plan; “Base” is untuned Qwen2.5-7B; “Groq” is gpt-oss-120b given the full rule book (mean of 3 runs, s.d. 2–4 points). The self-check bar covers the 12 requests where it was run.

4 Experimental Setup

Planner. We wrote 24 requests over three schemas (sales, zoo animals, IoT sensors), in the way people actually ask: “drop the cheap stuff”, “only EU region rows please”, a misspelled “filtr rows whre quanti > 10”, aggregations and numbered multi-stage requests. A plan is *executable* if it passes the structural and safety checks and every stage compiles to PySpark, and *meets intent* if the requested operations appear (patterns fixed in advance). It is *correct* if both hold. Raw and repaired outputs are scored on the same sample.

Live runs. All runs used an Azure for Students subscription (Data Factory copy, Databricks serverless, Blob storage) through the production API, with seeded CSVs of 1,000, 50,000 and 400,000 rows (0.06–25.5 MB). We downloaded every output and compared it with a reference computed locally from the same file. Estimates were logged before each run and only used earlier runs. Errors are MAPE. Local inference ran on an Apple M5 with 16 GB.

5 Results

5.1 Each part of the planner does a different job

Figure 3 shows that no single component is enough. The deterministic default always runs and never does what was asked. The untuned model understands a third of the requests but writes plans in the wrong shape, and repair cannot save them: the system falls back to the default plan in 15 of 24 cases. Fine-tuning lifts intent to 96%, yet only a third of its raw plans run. When a request names no containers, the model invents names Azure rejects, or wires a stage to read and write the same container. Repair fixes all of these without touching intent, and the self-check catches the one remaining miss, a missing aggregation, on its single re-plan.

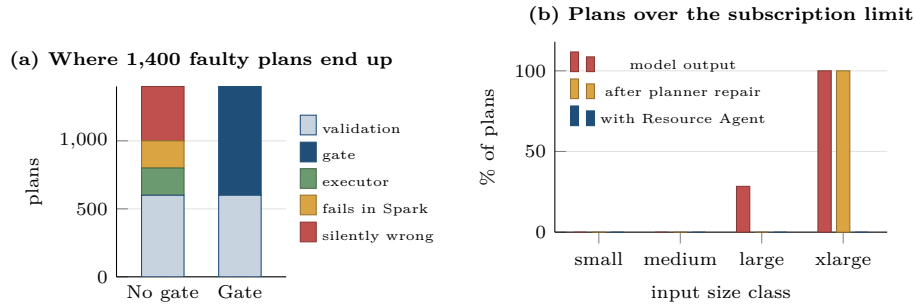


Fig. 4. (a) Seven kinds of fault injected into 200 valid plans each and followed through the real checks. Without the structural gate, 600 reach Databricks and 400 of those run “successfully” with wrong output. (b) Share of 1,000 plans that ask for more compute than the subscription allows.

The 120B cloud model behaves differently. With the whole rule book in its prompt, 75% of its raw plans already run, so repair adds little; it loses on intent. Some of that gap is a matter of style: it writes filters as `equals(region, 'EU')`, which our patterns do not count, so its numbers are a lower bound. Even so, the small local model is right far more often, costs nothing per plan and keeps the data on the machine. The cloud model is about twice as fast (19.7s against 41s per plan) at roughly a tenth of a cent.

5.2 Gates keep wrong plans out of the cloud

We took 200 valid plans and broke each one in seven ways, from a filter on a missing column to a code injection string, then followed every broken plan through the real checks in run order (Fig. 4a). The Manager’s own validation catches malformed plans, and the executor refuses code it cannot compile. But both are tolerant by design: an unknown stage type is skipped and an unsupported aggregation such as `median` is dropped. Without the gate, 43% of the faulty plans reach the cloud, and two thirds of those do not even fail. They just produce the wrong numbers. With the gate, none get through. It takes 0.07ms per plan.

Writing this harness also found a bug in the gate itself: type names inside `cast(x as double)` were read as columns, so 12.3% of valid plans were rejected. After a one-line configuration fix, 0 of 5,000 are. Compute sizing is a gate of its own (Fig. 4b). Even after the planner’s own size caps, every extra-large plan asks for more copy throughput than the student tier allows; with the Resource Agent, none do.

5.3 Live on Azure

Prediction. Looking through the feedback log, we noticed that the Resource Agent’s learned duration correction did nothing: every corrected notebook esti-

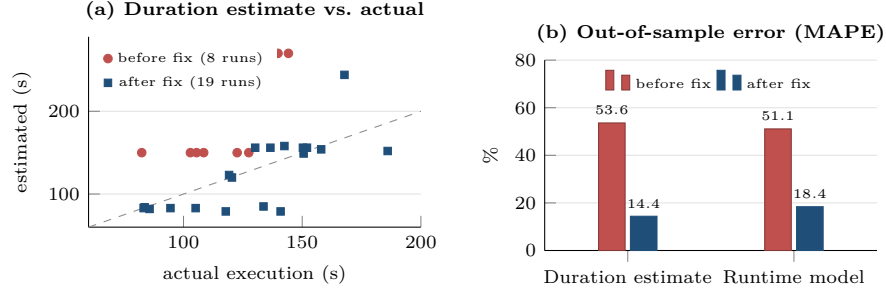


Fig. 5. Estimates made before each live run. After removing a cold-start floor that had blocked duration learning, the points move onto the diagonal. One run delayed 15 minutes inside Databricks is left out of the “after” set; with it, the errors are 18.3% and 22.0%.

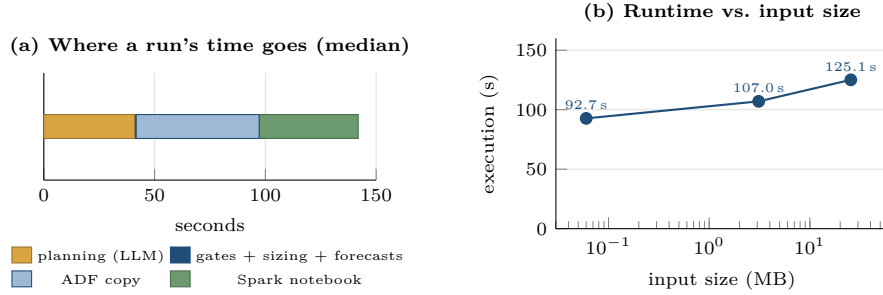


Fig. 6. (a) Median time per run over 14 live runs: the system’s own decisions take under a second; almost everything else is Azure start-up. (b) A 418× larger file takes only 35% longer.

mate was pushed back up to a fixed 120 s cold-start floor. Removing the floor, and learning one ratio per run and per size band, moved the estimates onto the diagonal (Fig. 5). On a fresh batch of 24 runs, duration error fell from 53.6% to 14.4% and runtime-model error from 51.1% to 18.4%. The runtime model improved because the duration estimate is its strongest input. That input is the “circular” feature from Sect. 3, and we tested dropping it: live error then rose from 36.5% to 142.1%. The feature is informative; the fix is to replace it with the Resource Agent’s settings, not to delete it.

Time. A request becomes verified output in two to three and a half minutes. Planning takes 29–53 s, and the run itself a median of 116 s. Of that, the system’s own decisions (validation, gates, sizing, forecasts, code generation) take less than one second; the rest is Azure copying data and starting Spark (Fig. 6). That is also why runtime barely depends on input size at this scale. Per pipeline, the system writes 108–195 lines of PySpark and 5–7 Data Factory objects that someone would otherwise have to write by hand.

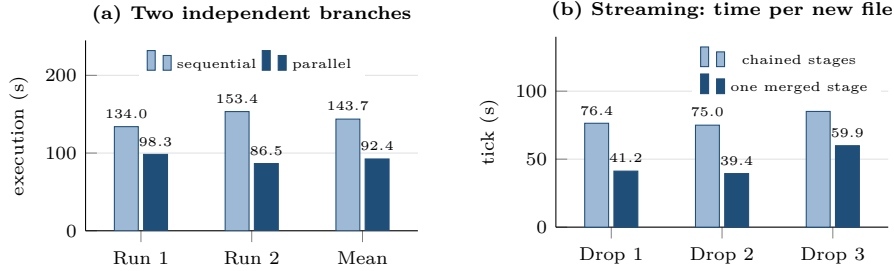


Fig. 7. (a) Running two independent notebook stages as one execution group saves about one Spark start-up (−36%). (b) Merging two streaming steps into one stage is 41% faster per new file, with identical output (285, 567 and 852 cumulative rows; about 300 expected per drop).

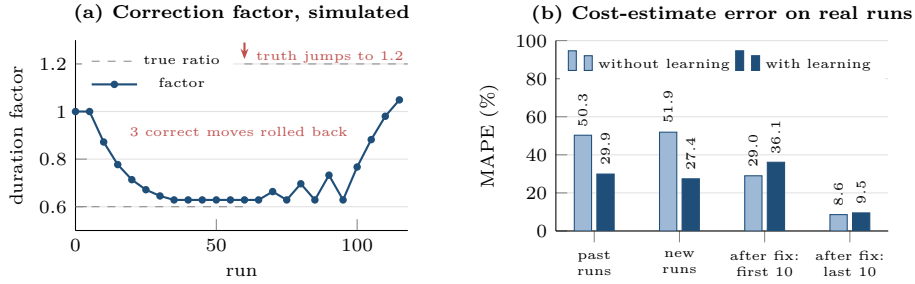


Fig. 8. (a) The learning agent’s real code on simulated runs: it converges to within 0.04 of the true ratio in about 35 runs, but after a sudden change its rollback rule reverts three correct moves. (b) Paired comparisons on the same real runs. After the duration fix, the old cost factor briefly hurt, until the loop relearned it.

Execution choices. Because start-up dominates, anything that avoids a second Spark job pays off (Fig. 7). Running two independent branches together cut 143.7 s to 92.4 s. For streaming, one merged stage handled each new file in 46.8 s instead of 78.8 s, and every file was processed exactly once.

Correct output. Of the 20 runs in the last batch that completed, 16 produced exactly the reference result. All four failures came from one plan, which we come back to in Sect. 5.5. Pinning the planner’s own settings instead of the Resource Agent’s made no measurable difference on the smaller files; on the 25 MB file the Resource Agent chose half the copy throughput and the copy took 81 s instead of 60 s. Its value is in staying within limits, not in speed.

5.4 The learning loop

On the same real runs, learned corrections cut cost-estimate error from 50.3% to 29.9% on past runs and from 51.9% to 27.4% on new ones (Fig. 8b). The live

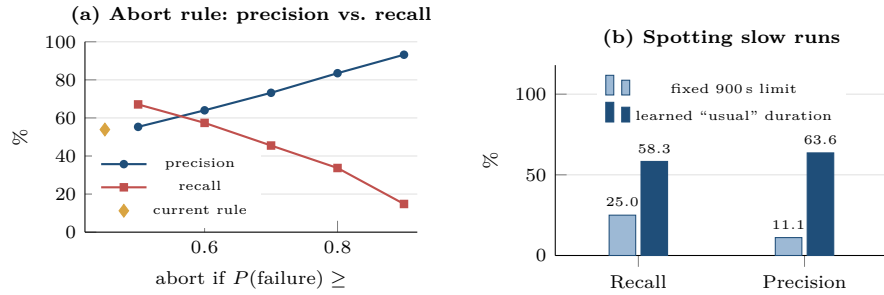


Fig. 9. (a) The Performance Agent aborts when “failure” is its most likely outcome, which is right only 53.9% of the time on held-out data (21,000 runs). A 0.7 threshold trades recall for 73.2% precision. (b) Flagging genuinely slow runs on four pipelines of very different size (simulated, 108 runs): a per-pipeline learned baseline beats one fixed limit.

batch also showed the loop adapting to another agent. The duration fix removed an over-estimate that the cost factor had been compensating for, so for the first ten runs the learned factor made things worse (36.1% against 29.0%). Over the next 20 runs the loop moved it from 0.795 to 1.106, and error came down to 9.5%. All nine of its reviews confirmed their change.

The rollback rule had to be fixed first. It used to count runs that were aborted before they started, whose “duration” of 0.1 s produced post-change errors of up to 68,094.7% and reverted two good updates. Simulation shows a subtler limit that remains (Fig. 8a): after the true ratio jumps, each correct move is judged against an error measured before the jump, looks worse, and is rolled back. Adaptation is delayed by about 30 runs.

5.5 What did not work

An abort rule that cries wolf. The Performance Agent stops a run when “failure” is the most likely of its three outcomes, even at $P \approx 0.5$. On held-out data that rule is right only 53.9% of the time (Fig. 9a). Live, it stopped plans that then completed correctly, once when the same stages were regrouped and once when the gate let the same plan through. A threshold of 0.7 would cut false aborts by 72%.

Completed is not correct. Four runs finished, passed every post-run check, and were wrong. The model had written one container name twice; the repair layer renamed the duplicate but remapped references by name, so one stage read and wrote the same container, and the aggregation counted raw and filtered rows together. Nothing flagged it except our output comparison. We reproduced the bug offline; the fix is to de-duplicate by position and to add a gate rule that no stage may read its own output.

Playing it safe can mean doing nothing. The Cost Agent only applies a cheaper configuration if it can show the run will stay within a learned deadline. In 20 live runs it never could, so it never made a run worse, and never saved a cent either.

6 Discussion and Limitations

One pattern runs through the results: the learned components are good at proposing and bad at deciding. Fine-tuning gives the planner its understanding, but repair makes the plans run; the gate, not the model, keeps bad plans out of the cloud; and learned corrections help only behind evidence gates and rollback. The one place a learned component decided alone, the abort rule, it was wrong almost half the time.

The evidence has limits. Live runs used one student subscription, files up to 25.5 MB, at most three stages and two to four repeats per configuration. The sizing, runtime and cost models are trained mostly on synthetic data, calibrated to real telemetry where we had it, and “actual cost” is our cost formula at the measured runtime, not an Azure invoice. The planner was tested on 24 requests scored by fixed patterns, which undercount correct plans written in other styles. We count the artefacts a person would otherwise write, but we have not yet timed people doing it.

7 Conclusion

A small model that runs on a laptop, wrapped in deterministic repair and checked by gates, turned plain-English requests into correct cloud pipelines more reliably than a model seventeen times its size, and a guarded learning loop kept its forecasts honest on real runs. The failures taught us as much: learned gates need calibrated thresholds, and a pipeline that completes has not necessarily done what was asked. Next, we will time engineers on the same tasks, fix the two bugs described here, and build output checks into every run.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Kephart, J.O., Chess, D.M.: The vision of autonomic computing. *Computer* **36**(1), 41–50 (2003)
2. Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W.: LoRA: Low-rank adaptation of large language models. In: *International Conference on Learning Representations* (2022)
3. Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L.: QLoRA: Efficient fine-tuning of quantized LLMs. In: *Advances in Neural Information Processing Systems*, vol. 36 (2023)

4. Qwen Team: Qwen2.5 technical report. arXiv preprint arXiv:2412.15115 (2024)
5. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: ReAct: Synergizing reasoning and acting in language models. In: International Conference on Learning Representations (2023)
6. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S.: Reflexion: Language agents with verbal reinforcement learning. In: Advances in Neural Information Processing Systems, vol. 36 (2023)
7. Madaan, A., Tandon, N., Gupta, P., et al.: Self-Refine: Iterative refinement with self-feedback. In: Advances in Neural Information Processing Systems, vol. 36 (2023)
8. Huang, J., Chen, X., Mishra, S., Zheng, H.S., Yu, A.W., Song, X., Zhou, D.: Large language models cannot self-correct reasoning yet. In: International Conference on Learning Representations (2024)
9. Wu, Q., Bansal, G., Zhang, J., et al.: AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv preprint arXiv:2308.08155 (2023)
10. Hong, S., Zhuge, M., Chen, J., et al.: MetaGPT: Meta programming for a multi-agent collaborative framework. In: International Conference on Learning Representations (2024)
11. Yu, T., Zhang, R., Yang, K., et al.: Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In: Proceedings of EMNLP, pp. 3911–3921 (2018)
12. Rajkumar, N., Li, R., Bahdanau, D.: Evaluating the text-to-SQL capabilities of large language models. arXiv preprint arXiv:2204.00498 (2022)
13. Van Aken, D., Pavlo, A., Gordon, G.J., Zhang, B.: Automatic database management system tuning through large-scale machine learning. In: Proceedings of SIGMOD, pp. 1009–1024 (2017)